1. Introduction:

Purpose of the project: The objective of this project is to create a real-time group chat application using Java. It allows multiple users to communicate together over a network and provides a user-friendly interface similar to modern chat apps. This project demonstrates real-world applications of Java's client-server communication, GUI design, and multithreading.

Problem statement: This project solves the problem of instant messaging within the network, where users can send and receive messages without relying on any third-party messaging service. It ensures effective communication between multiple users, maintains message formatting, and guarantees message delivery in real-time.

Technologies Used: This project uses Java to implement various technologies. Java Swing is used to create GUIs, Java Sockets for network communication, and multithreading to handle multiple clients at the same time. Utilities like Calendar and SimpleDateFormat have also been used to show the time of the message.

Key features:

- Real-time messaging between multiple clients.
- User-friendly interface with message bubbles, timestamps, and sender names.
- Draggable, undecorated windows for a modern look.
- Possibility to add multimedia features (camera, gallery icons, etc.) in the future.
- Using separate threads for each client results in no lag or delay in communication.

2. Objectives: The main objective of this project is to create a real-time chat application that ensures seamless communication between multiple users over a network. This app is designed to allow users to instantly exchange messages in an easy-to-use and interactive interface, where all information, including the sender's name and time, is displayed.In addition, the project focuses on building a robust client-server architecture using Java Sockets and multithreading, so that multiple users can connect at the same time and communicate without any delays. The application has been designed with future extensibility and the possibility of adding additional features such as multimedia support, Which establishes it as an effective solution for exchanging messages on local networks.

Project Description/Implementation:

- **Constructor UserOne():**

UserOne() sets up the entire window's GUI and its functionality.The main JFrame (f) is created with the specified size, location, background color, and layout set to null.A header panel (p1) is created, with a black background and absolute positioning.Various icons are added: back button (exit), profile picture, video call, phone call, and more options. Each ImageIcon is loaded, resized, and added to a JLabel.Name and status labels are added to the header panel.A chat area panel (a1) is created where messages will be displayed.A text field and Send button are added at the bottom, and an action listener is given to the Send button.An additional icons panel (Plus, Camera, Gallery) is created and added to the left side.

Imported Statements: These import statements help to use various GUI components, such as JTextField, JButton, JLabel, JPanel, JFrame, BoxLayout, and ImageIcon.The AWT classes (Font, Color, Image, Point, BorderLayout, FlowLayout) are used to manage layout, design, positioning, and color.Event-handling classes (MouseEvent, MouseAdapter, ActionListener, ActionEvent, MouseMotionAdapter) are used to handle button click, mouse click, and drag events.Calendar and SimpleDateFormat are used to display time, EmptyBorder is used to set spacing around labels, Socket is used for network communication, and BufferedReader/BufferedWriter is used to handle input-output operations.

This figure shows Swing, AWT, Event, Calendar, Empty Border, Socket, IO imported files.

```
//All swing Files
import javax.swing.JTextField;
import javax.swing.JButton;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JFrame;
import javax.swing.Box;
import javax.swing.BoxLayout;
import javax.swing.ImageIcon;

//AWT files
import java.awt.Font;
import java.awt.Color;
import java.awt.Image;
import java.awt.Point;
import java.awt.BorderLayout;
import java.awt.FlowLayout;

//Event
import java.awt.event.MouseEvent;
import java.awt.event.MouseAdapter;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
import java.awt.event.MouseMotionAdapter;

//For calender
import java.util.Calendar;
import java.text.SimpleDateFormat;

// For Empty Border
import javax.swing.border.EmptyBorder;

// For Socket
import java.net.Socket;

//input & output
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.DataOutputStream;
import java.io.OutputStreamWriter;
import java.io.InputStreamReader;
```

Figure no: 01

3. GUI Implementation

This figure shows a JPanel which has used for the Top header panel (Using Black)

```
JPanel p1 = new JPanel();
p1.setBackground(Color.BLACK);
p1.setBounds(x: 0, y: 0, width: 450, height: 70);
p1.setLayout(mgr: null);
f.add(p1);
```

Figure no: 02

Top Panel (Header) Creation: This section creates a top panel (p1), which will serve as the top header of the JFrame. The panel's background is set to black, its size is 450×70 pixels, and it is positioned at the top-left corner of the frame (0,0). setLayout(null) is used to position the components inside this panel (profile picture, back button, call/video icon, name, and status) using absolute coordinates.Finally, this panel is added to the main frame using f.add(p1).

Back Button Implementation: In this section, a "Back" icon button has been added to the top-left of the frame.

This figure shows Image for 3 Dot Icon

```
ImageIcon i1 = new ImageIcon(ClassLoader.getSystemResource(name: "icons/3.png"));
Image i2 = i1.getImage().getScaledInstance(width: 25, height: 25, Image.SCALE_DEFAULT);
ImageIcon i3 = new ImageIcon(i2);
JLabel back = new JLabel(i3);
back.setBounds(x: 5, y: 20, width: 25, height: 25);
p1.add(back);

back.addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent ae) {
        System.exit(status: 0);
    }
});
```

Figure no: 03

Explanation: First, the icons/3.png image is loaded using ImageIcon. Then the image is resized to 25×25 pixels using getScaledInstance, and then converted back to ImageIcon.This image is set to a JLabel, which acts as a back button. It is placed at a specific position in the frame using setBounds.The label is then added to the header panel using p1.add(back).A MouseListener is added to handle the click event. When the user clicks the back icon, System.exit(0) is called and the entire program is terminated.

Profile Icon / Image Logic:

This figure shows a Image Icon for DIU logo

```
ImageIcon i4 = new ImageIcon(ClassLoader.getSystemResource( name: "icons/mirzapur.png"));
Image i5 = i4.getImage().getScaledInstance( width: 60, height: 60, Image.SCALE_DEFAULT);
ImageIcon i6 = new ImageIcon(i5);
JLabel profile = new JLabel(i6);
profile.setBounds( x: 40, y: 5, width: 60, height: 60);
p1.add(profile);
```

Figure no: 04

In this part, the user's profile picture is added to the top panel.First, the image file icons/mirzapur.png is loaded from the project resources using ImageIcon. This step follows the same logic as loading the back button icon.Next, the image is resized to 60×60 pixels using getScaledInstance, as the profile picture should be larger than the other icons.After resizing, the image is converted back to an ImageIcon so that it can be set in the JLabel.A JLabel is created called profile, and the image is positioned on the top panel using profile.setBounds(40, 5, 60, 60) to maintain proper spacing and alignment with the other icons.Finally, the label is added to the panel using p1.add(profile).

The logical flow for adding a profile picture is:

Load Image → Resize → Convert → JLabel → Position → Add to Panel.

This same pattern is repeated for all icons in the top panel, such as the video, phone, and back icons.

Name and Status Labels: In this part, the username and status are added to the top panel (p1).A JLabel is created with the name name, and the text is set to "DIU". It is positioned at coordinates (110, 15), has a width of 100 pixels, and a height of 18 pixels. Its font is SAN_SERIF, bold, size 18, and the text color is set to white.Similarly, a JLabel is created called status, which displays additional information ("Arafat, Alif, Sagor, Shakib, Akash"). It is positioned at coordinates (110, 35), has a width of 160 pixels and a height of 18 pixels, using the same font family but size 14.Both labels are added to the header panel using p1.add().These labels clearly display the user's identity and status information in line with the profile picture in the user interface.

This figure shows the group name DIU or may be used combine study

```
JLabel name = new JLabel( text: "DIU");
name.setBounds( x: 110, y: 15, width: 100, height: 18);
name.setForeground(Color.WHITE);
name.setFont(new Font( name: "SAN_SERIF", Font.BOLD, size: 18));
p1.add(name);

JLabel status = new JLabel( text: "Arafat, Alif, sagor,Shakib, Akash");
status.setBounds( x: 110, y: 35, width: 160, height: 18);
status.setForeground(Color.WHITE);
status.setFont(new Font( name: "SAN_SERIF", Font.BOLD, size: 14));
p1.add(status);
```

Figure no: 05

Video, Phone, and More Icons:

This figure shows 3 icons for Video, phone, Back arrow sign.

```
ImageIcon i7 = new ImageIcon(ClassLoader.getSystemResource(name: "icons/video.png"));
Image i8 = i7.getImage().getScaledInstance(width: 30, height: 30, Image.SCALE_DEFAULT);
ImageIcon i9 = new ImageIcon(i8);
JLabel video = new JLabel(i9);
video.setBounds(x: 300, y: 20, width: 30, height: 30);
p1.add(video);

ImageIcon i10 = new ImageIcon(ClassLoader.getSystemResource(name: "icons/phone.png"));
Image i11 = i10.getImage().getScaledInstance(width: 35, height: 30, Image.SCALE_DEFAULT);
ImageIcon i12 = new ImageIcon(i11);
JLabel phone = new JLabel(i12);
phone.setBounds(x: 360, y: 20, width: 35, height: 30);
p1.add(phone);

ImageIcon i13 = new ImageIcon(ClassLoader.getSystemResource(name: "icons/3icon.png"));
Image i14 = i13.getImage().getScaledInstance(width: 10, height: 25, Image.SCALE_DEFAULT);
ImageIcon i15 = new ImageIcon(i14);
JLabel morevert = new JLabel(i15);
morevert.setBounds(x: 420, y: 20, width: 10, height: 25);
p1.add(morevert);
```

Figure no: 06

In this section, video call, phone call, and “more options” icons are added to the top panel (p1).First, the video icon (icons/video.png) is loaded using ImageIcon, resized to 30×30 pixels, converted back to ImageIcon, and displayed in a JLabel positioned at coordinates (300, 20). Next, the phone icon (icons/phone.png) is loaded, resized to 35×30 pixels, transformed, and placed in a JLabel at coordinates (360, 20).Finally, the “more options” vertical icon (icons/3icon.png) is loaded, resized to 10×25 pixels, transformed, and positioned in the JLabel at coordinates (420, 20).Each label is added to the header panel using p1.add().These icons provide access to callouts and additional options, keeping the header visually organized by matching the profile picture and name/status labels.

Message Input Text Field: In this section, a text field is added to the main frame (f) for typing messages.A JTextField is created named text, and the initial text is set to “Message”. It is located at coordinates (130, 650), has a width of 200 pixels, and a height of 32 pixels.Its background color is set to dark gray, text color to white, and font is set to SAN_SERIF, plain, size 16.Finally, the text field is added to the frame using f.add(text).This provides an input area for the user to type a message into the chat interface.

This figure shows an abstract text message that is for simplicity.

```
text = new JTextField(text: "Message");
text.setBounds(x: 130, y: 650, width: 200, height: 32);
text.setBackground(Color.DARK_GRAY);
text.setForeground(Color.WHITE);
text.setFont(new Font(name: "SAN_SERIF", Font.PLAIN, size: 16));

f.add(text);
```

Figure no: 07

Send Button and Frame Configuration: In this part, a “Send” button is added to the main frame (f).A JButton is created with the name send, label “Send” and position (350, 650), width 80 pixels, height 32 pixels.Its background color is set to blue, text color to white, and font is set to SAN_SERIF, bold, size 16.An ActionListener is attached using send.addActionListener(this) to handle click events when the user sends a message.The button is then added to the frame using f.add(send).Additionally, the main frame is set to 450×700 pixels in size and positioned at (950, 50) on the screen. The frame is made undecorated using setUndecorated(true), which removes the default window borders, and sets the background color to black.These settings create a beautiful, custom-styled chat window.

This figure shown as Send Button and Frame Configuration.

```
JButton send = new JButton(text: "Send");
send.setBounds(x: 350, y: 650, width: 80, height: 32);
send.setBackground(Color.BLUE);
send.setForeground(Color.WHITE);
send.addActionListener(this);
send.setFont(new Font(name: "SAN_SERIF", Font.BOLD, size: 16));
f.add(send);

f.setSize(width: 450, height: 700);
f.setLocation(x: 950, y: 50);
f.setUndecorated(undecorated: true);
f.getContentPane().setBackground(Color.BLACK);
```

Figure no: 08

Extra Icons Panel (Plus, Camera, Gallery): In this section, some more icons are created for additional functionality and prepared for the user interface.The "Plus" icon (icons/Plus.png) is loaded using ImageIcon, resized to 30×30 pixels, converted back to ImageIcon, and displayed in a JLabel named plus.Similarly, the "Camera" icon (icons/Camera.png) is loaded, resized to 30×30 pixels, transformed, and displayed in a JLabel named camera.The "Gallery" icon (icons/Gallery.png) follows the same process and is displayed in a JLabel named gallery.These icons give the user quick access to additional features and can later be added to a separate panel on the left side of the chat interface.

This figure shown as Extra Icons Panel **.**

```
// Plus icon
        ImageIcon iPlus = new ImageIcon(ClassLoader.getSystemResource(name: "icons/Plus.png"));
        Image imgPlus = iPlus.getImage().getScaledInstance(width: 30, height: 30, Image.SCALE_DEFAULT);
        ImageIcon plusIcon = new ImageIcon(imgPlus);
        JLabel plus = new JLabel(plusIcon);

// Camera icon
        ImageIcon iCamera = new ImageIcon(ClassLoader.getSystemResource(name: "icons/Camera.png"));
        Image imgCamera = iCamera.getImage().getScaledInstance(width: 30, height: 30, Image.SCALE_DEFAULT);
        ImageIcon cameraIcon = new ImageIcon(imgCamera);
        JLabel camera = new JLabel(cameraIcon);

// Gallery icon
        ImageIcon iGallery = new ImageIcon(ClassLoader.getSystemResource(name: "icons/Gallery.png"));
        Image imgGallery = iGallery.getImage().getScaledInstance(width: 30, height: 30, Image.SCALE_DEFAULT);
        ImageIcon galleryIcon = new ImageIcon(imgGallery);
        JLabel gallery = new JLabel(galleryIcon);
```

Figure no: 09

Left Panel for Extra Icons: In this section, a JPanel is created called Left, which will hold the additional icons (Plus, Camera, Gallery). The panel uses a null layout (new JPanel(null)) so that the components can be positioned correctly using setBounds.The panel is positioned at coordinates (10,650), has a width of 140 pixels and a height of 40 pixels, and the background color is set to black.Each icon label determines its position within the panel: the Plus icon at (0, 0), the Camera icon at (40, 0), and the Gallery icon at (80, 0), each with a size of 30×30 pixels. Finally, the icons are added to the panel using Left.add().This panel neatly organizes the additional functionality icons on the left side of the message input area, ensuring a clean and intuitive user interface.

This figure is shown as an empty panel

```
JPanel Left = new JPanel(layout: null); // null layout to use setBounds
Left.setBounds(x: 10, y: 650, width: 140, height: 40);
Left.setBackground(Color.BLACK);

plus.setBounds(x: 0, y: 0, width: 30, height: 30);
camera.setBounds(x: 40, y: 0, width: 30, height: 30);
gallery.setBounds(x: 80, y: 0, width: 30, height: 30);

Left.add(plus);
Left.add(camera);
Left.add(gallery);
```

Figure no: 10

Displaying the Frame: Finally, the parent frame (f) is made visible by calling f.setVisible(true). It displays all the components added to the frame, including the header panel, profile picture, icons, chat area, text field, Send button, and additional icon panel on the screen, allowing the user to interact with the entire chat interface. Without this step, the GUI will be created in memory but not displayed on the screen.

- .setVisible(true);
-

Socket and I/O Stream Initialization: In this part, a socket connection is established on port 2003 to the server running on localhost. A Socket object is created for this connection, named socket.To send data to the server, a BufferedWriter is created called writer, and it is connected to the socket's output stream using OutputStreamWriter. Similarly, to receive data from the server, a BufferedReader is created called reader, and it is connected to the socket's input stream using InputStreamReader.The entire process is placed in a try-catch block to handle possible exceptions, and if any errors occur, they are printed using e.printStackTrace().This setup allows the chat application to communicate in real-time over the network.

This figure is shown as Socket features Host and port number

```
try {
    Socket socket = new Socket(host: "localhost", port: 2003);
    writer = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream()));
    reader = new BufferedReader(new InputStreamReader(socket.getInputStream()));
} catch (Exception e) {
    e.printStackTrace();
}
```

Figure no: 11

Making the JFrame Draggable: In this part, the facility to drag the undecorated frame (f) onto the screen is added.A MouseListener is added to the frame to detect when the mouse is pressed and the initial click point is stored in mouseDownCompCoords.Next, a MouseMotionListener is added to track mouse drag events.When dragging, the current position of the mouse on the screen is taken and the frame position is updated using the coordinates of the initial click.This allows the user to click anywhere in the frame and drag it smoothly, which compensates for the lack of a default window title bar in undecorated frames.

This figure shown as the code for Mouse dragger

```
    //Make JFrame draggable
    f.addMouseListener((MouseAdapter) mousePressed(e) → {
        mouseDownCompCoords = e.getPoint();  // Record click point
    });

    f.addMouseMotionListener((MouseMotionAdapter) mouseDragged(e) → {
        Point currCoords = e.getLocationOnScreen();
        f.setLocation( x: currCoords.x - mouseDownCompCoords.x,  y: currCoords.y - mouseDownCompCoords.y);
    });
}
```

Figure no: 12

Implementation Details:

- Implemented using MouseListener and MouseMotionListener.
- The coordinates of the initial mouse click (mouseDownCompCoords) are stored and the frame position is updated while dragging.
- Not using the default window decorations (setUndecorated(true)) provides a modern, desktop-like user experience.

4. Methods:

4.1 Sending Messages – actionPerformed Method : This method is executed when the user clicks the Send button. It performs several tasks: displaying the message in the chat window and sending the message to the server.An HTML string is created by taking the text (text.getText()) from the input field and appending it to the user's name.

This figure shown as actionPerformed Method.

```
public void actionPerformed(ActionEvent ae) {
    try {
        String out = "<html><p>" + name + "</p><p>" + text.getText() + "</p></html>";

        JPanel p2 = formatLabel(out);

        a1.setLayout(new BorderLayout());

        JPanel right = new JPanel(new BorderLayout());
        right.add(p2, BorderLayout.LINE_END);
        right.setBackground(Color.BLACK);
        vertical.add(right);
        vertical.add(Box.createVerticalStrut(height: 15));

        a1.add(vertical, BorderLayout.PAGE_START);

        try {
            writer.write(out);
            writer.write(str: "\r\n");
            writer.flush();
        } catch (Exception e) {
            e.printStackTrace();
        }

        text.setText(t: "");

        f.repaint();
        f.invalidate();
        f.validate();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Figure no: 13

- **Creating a message panel:** The formatLabel(out) method places the formatted message into a JPanel (p2), with the correct styling, padding, and background color, so that it looks like a chat bubble.
- **Aligning the message to the right:** A new JPanel is created called right and its layout is set to BorderLayout. The formatted message panel (p2) is added to the line-end (right), indicating that this is a message sent by the user. The background of the right panel is kept black to match the chat interface.
- **Adding vertical spacing:** The message panel is added to a vertical box layout (Box object) that arranges the components vertically. A 15-pixel vertical strut is added below the message to separate it from the next message.
- **Added to the chat area:** The vertical box is added to the main chat panel (a1) in BorderLayout.PAGE_START, so that new messages are sorted sequentially from top to bottom.
- **Sending messages to the server:** The formatted message is sent to the socket using BufferedWriter (writer). Each message ends with \r\n which indicates a line break, and flush() sends the message immediately.
- **Resetting the input field:** After sending the message, the input field is cleared with text.setText("") so that the user can type a new message.
- Refreshing the GUI: The frame is repainted and revalidated using f.repaint(), f.invalidate(), and f.validate(), so that the newly added message appears on the display.
- **Exception Handling:** All operations are placed inside try-catch blocks, so that any I/O errors or runtime exceptions can be safely handled. In case of errors, a stack trace is printed.
-

4.2 Message Formatting – formatLabel Method: The formatLabel method is responsible for creating a styled message panel to display in the chat interface. It takes a string (out) as input and returns a JPanel that contains the formatted message. First, a JPanel is created with a black background and a layout of BoxLayout.Y_AXIS (to arrange the components vertically). Then a JLabel is created, named output, which displays the message text. The text is placed in HTML and given a specific width (150px) so that the words are properly separated at line breaks.

This figure shows formatLabel for better structure the text format

```java
public static JPanel formatLabel(String out) {
    JPanel panel = new JPanel();
    panel.setBackground(Color.BLACK);
    panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));

    JLabel output = new JLabel("<html><p style=\"width: 150px\">" + out + "</p></html>");
    output.setFont(new Font(name: "Tahoma", Font.PLAIN, size: 16));
    output.setBackground(Color.ORANGE);
    output.setOpaque(isOpaque: true);
    output.setBorder(new EmptyBorder(top: 0, left: 5, bottom: 0, right: 50));

    panel.add(output);

    Calendar cal = Calendar.getInstance();
    SimpleDateFormat sdf = new SimpleDateFormat(pattern: "HH:mm");

    JLabel time = new JLabel();
    time.setText(sdf.format(cal.getTime()));
    time.setForeground(Color.WHITE);

    panel.add(time);

    return panel;
}
```

Figure no: 14

Next, the current time is taken using Calendar and formatted using SimpleDateFormat in the format "HH:mm". A separate JLabel named time displays this timestamp in white text below the message. This label is also added to the panel. Finally, a fully formatted JPanel containing both the message and its timestamp is returned. This panel can then be added to the chat area (a1) and properly aligned to the left or right depending on whether the message is incoming or outgoing.

4.3 Receiving Messages – run Method: The run method handles incoming messages from the server in real-time and displays them in the chat interface. It continuously reads messages from the BufferedReader (reader) in an infinite while (true) loop. For each message, it is first checked whether the message contains the user's own name; if it does, it is skipped, so that the user's own outgoing message is not shown again. For incoming messages, the method calls formatLabel(msg) to create a styled JPanel that contains the message text and timestamp. A new JPanel is then created named left and the layout is BorderLayout, with the background set to black to match the chat interface. The formatted message panel is added to the line-start of the left panel, indicating that it is an incoming message. This panel is added to the vertical box so that messages are arranged sequentially from top to bottom. The vertical box is then added to the main chat area (a1) in BorderLayout.PAGE_START. After each addition, the frame is refreshed using repaint(), invalidate(), and validate(), so that new messages can be displayed immediately. The entire process is enclosed in a try-catch block, so that any I/O exceptions can be safely handled.

This figure shows the run method for exception handling

```
public void run() {
    try {
        String msg = "";
        while (true) {
            msg = reader.readLine();
            if (msg.contains(name)) {
                continue;
            }

            JPanel panel = formatLabel(msg);

            JPanel left = new JPanel(new BorderLayout());
            left.setBackground(Color.BLACK);
            left.add(panel, BorderLayout.LINE_START);
            vertical.add(left);

            a1.add(vertical, BorderLayout.PAGE_START);

            f.repaint();
            f.invalidate();
            f.validate();
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Figure no: 15

The vertical box is then added (BorderLayout.PAGE_START) above the main chat area (a1). After each addition, the frame is refreshed using repaint(), invalidate(), and validate(), so that new messages are displayed immediately. The entire process is enclosed in a try-catch block, so that any I/O exceptions can be safely handled.

5. Server part

Introduction:

Communication software has become an essential part of modern computing. Applications like WhatsApp, Messenger, and Slack use server-client architecture to enable real-time messaging between users. In this project, a simplified version of such a system has been developed using Java. The project builds a chat server using Java's networking API. The server listens for clients, accepts connections, receives messages, and broadcasts them to all connected users. Communication is reliable and sequential because of the use of TCP sockets.

This project highlights some important concepts, such as:

- ❖ ServerSocket and Socket Communication
- ❖ Multithreading
- ❖ Input/output stream handling
- ❖ Maintaining multiple clients using data structures

The importance of this part:

1. Supports multi-client communication: Many users can connect and chat at the same time.
2. Enables real-time messaging: When a client sends a message, it is immediately forwarded to everyone.
3. Uses threads for parallel communication: Each client runs on a separate thread, which helps the server handle many clients simultaneously.
4. Practical Example of Java Socket Programming: This code teaches how a server and client communicate over the network.
5. Broadcasting demonstrates: A message can be sent to all connected users at once, which is a key feature of chat applications.

Working Process:

1. Server Startup :The server starts by creating a ServerSocket on port 2003. This opens the server for incoming connections.
2. Client Acceptance:When a client connects, the server accepts it and creates a new Socket object. A separate thread is created for each connected client, allowing for parallel communication.
3. Creating a stream:

   For each client:

   - ❖ BufferedReader receives messages,BufferedWriter sends outgoing messages
4. Adding clients to the list:Each client's writer is stored in a Vector, so that the server can send messages to all users.
5. Reading messages:The server continuously reads each incoming message using readLine().
6. Broadcasting:Each received message is looped through a Vector and sent to all clients.
7. Persistent Communication:As long as the client is connected, the server remains active.

Project Description/Implementation:

Server class structure:

The Server class implements Runnable, which means each client connection runs on its own thread. This class also maintains a shared list, called clients, which stores the output streams of all connected users. This shared Vector allows the server to send messages to each currently connected user.

```
Socket socket;

public static Vector clients = new Vector();
```

Purpose of the constructor: The constructor takes a Socket object, which belongs to the newly connected client.This socket is stored so that the server can read and send messages from that client.An exception is printed if there is a problem during initialization. Basically, the constructor's job is to assign the passed socket to the instance.

```
public Server(Socket socket) {

 this.socket = socket;

                    }
```

What Happens Inside run():

```
BufferedReader reader = new BufferedReader(

      new InputStreamReader(socket.getInputStream())

);

BufferedWriter writer = new BufferedWriter(

      new OutputStreamWriter(socket.getOutputStream())

);
```

The run() method is where the real work of communication happens.

Two streams are created here:

- BufferedReader: for receiving text from the client.
- BufferedWriter: for sending text to the client.

When the Writer is created, it is added to the shared clients list.This means that the server remembers how to contact this user when broadcasting messages.

Receiving and redistributing messages: The server is in an endless loop, waiting for messages from connected clients.Whenever a line of text arrives, the server prints it to its console.

```
String message = reader.readLine().trim();

System.out.println("Received: " + message);
```

After that, it forwards the message to every client stored in the Clients list: This broadcasting step is what turns the program into a group chat system. Every connected user receives the same message, regardless of who sent it.

```
for (int i = 0; i < clients.size(); i++) {

   BufferedWriter bw = (BufferedWriter)
clients.get(i);

   bw.write(message + "\r\n");

   bw.flush();

}
```

6. Conclusion :

The Group Chat Application developed using Java Swing and TCP Socket successfully demonstrates a real-time multi-user communication system. Although this report includes the code for a single user interface and the server module, the actual project was implemented with four separate user clients that connect simultaneously to the same server. Each user interacts with the system independently, and the server broadcasts messages to all connected clients, allowing smooth group communication.

During the development of this project, we implemented essential networking concepts such as socket programming, input/output stream handling, and multithreaded communication. The server operates using multiple threads, enabling it to manage multiple clients concurrently without interruption. On the client side, Java Swing was used to design an attractive and responsive chat interface that updates messages dynamically in real time.

This project enhanced our understanding of client–server architecture, message broadcasting, event-driven UI programming, and concurrency management in Java. It also strengthened our practical problem-solving skills by dealing with challenges such as continuous data streaming, UI synchronization, thread handling, and connection management.

Overall, the project achieves its objective of creating a functional, interactive, and real-time group chat platform. The experience gained from designing and implementing this system will significantly benefit future work in network-based applications, distributed systems, and advanced Java software development.

❑ CEA (Complex Engineering Problems)

P1 – Depth of Knowledge (K3, K5, K6):

To build the Group Chat Application using **Java Swing UI** and **TCP Socket networking**, we required strong engineering knowledge in multiple areas:

- **K3 – Theory-based Engineering Knowledge:**
  We applied theoretical concepts of
  - Java networking (Socket, ServerSocket)
  - Input/Output streams
  - Multithreading
  - Swing event-driven programming
    This theoretical base was essential to design real-time message communication between client and server.

K5 – Engineering Design:
Before development, we designed the overall structure of the system:

- Designed separate modules for UI, client handler, server handling
- Planned how multiple clients will communicate through a central server
- Created wireframe of the chat interface (header panel, message area, typing area) This design thinking helped us create a stable and scalable group chat application.

K6 – Engineering Practice:
The project applies practical engineering skills by implementing real-time chat communication over TCP using multiple threads.
We handled:

- Network exceptions
- Connection drops
- Message broadcasting
- UI updates from different threads
  These practices match real-world applications like WhatsApp Web or Messenger desktop clients.

P3 – Depth of Analysis:

What guidance/ constraints were given to develop the solution/ design?

There might be multiple solutions

Students may have been given clear boundaries and specific details of what they had to do, or they may have had to define some or all of the boundaries to the problem themselves and work with **limited**
information to decide how the work should be carried out

The problem may have been the one that they regularly encountered, but with slight case-specific variations

P7: Interdependence:

The problem is able to be broken down into smaller components or sub-problems

❑ CEA (Complex Engineering Activities)

A1 – Range of Resources:

To build the application, we had to learn new tools and resources, including:

- Java Networking tutorials (Socket, ServerSocket)
- Swing Layout Management (BorderLayout, BoxLayout, Panels)
- YouTube tutorials
- ChatGPT guidance
- Google search guidance
- GFG websites
- W3 School
- Oracle Java Documentation
  These diverse technical resources were essential to complete the project.

A5: Familiarity:

To what extent is the previous experiences routinely encountered and resolved using well understood practices?

The experience is a:

- New experience which is not previously or only rarely encountered.
- Familiar experience with some (or many) unique issues that made
  The communication difficulty level increased.